



**BITS Pilani**  
K K Birla Goa Campus

## **Final Report**

By Sreeram Padmesh  
2022A4PS1459G

# **Table of Contents**

## **1. Abstract**

## **2. Introduction**

- 2.1 Ackermann Steering System
- 2.2 Differential Drive System

## **3. Objective**

## **4. Components Used**

- 3.1 Arduino Uno
- 3.2 Servo Motor
- 3.3 Ultrasonic Sensor
- 3.4 Color Sensor

## **5. CAD Drawing**

## **6. Methodology**

## **7. Electric Circuit**

## **8. Circuit Picture**

## **9. Python Code Explanation**

## **10. Conclusion**

## **Abstract**

This project focuses on the design and implementation of a differential-drive autonomous self-parking robot which can identify and occupy designated parking areas using a color sensor. The robot will be programmed in C++ on the Arduino Uno platform, which will be the main micro-controller for all hardware components. The drive system is built on four TT gear motors coupled with wheels, powered by two 18650 Li-ion batteries, and managed through a Motor Driver Shield for the control of motion. Obstacle avoidance and distance measurement will be achieved using an ultrasonic sensor, while a servo motor will be employed to provide directional control for the sensor system.

The project involves the TCS3200 color sensor, which will monitor the surface below the robot for specific color markers that indicate a parking location. The robot will initiate the parking process when the appropriate color pattern is recognized, using its differential drive construction and real-time sensor feedback to accurately position itself while parking. This project aims to combine color-based detection of an environment with ultrasonic sensing of obstacles to show a low-cost solution to autonomous parking.

## **Introduction**

Autonomous vehicles are becoming a big part of the automotive industry and while some cars are being built to work fully autonomously, there are some vehicles which have certain specific tasks that can be done autonomously. One of them is parking. While most real world cars use a drive system called Ackermann Steering System, I will be using a simpler one called differential drive system. A more detailed explanation of both is given below for better understanding of why the latter was chosen.

### **Ackermann Steering System**

Ackermann steering is a common choice in vehicles, and larger robotic systems where prudent steering geometry is a functionality needed. In Ackermann steering, both front wheels pivot at differing angles to equal out the difference in radii when turning to allow for all wheels to roll freely without slippage. This is a desirable configuration for a vehicle that runs at speed or requires accurate cornering with little roll drag. But for small robotic prototypes, using ackermann steering may not be mechanically feasible as it incorporates linkages and precise steering actuation to overcome the precise use of the two front wheels, and typically requires more powerful servos to operate in a reliable manner since routing the steering forces, for the front wheels require them to be able to handle a specific amount of braking. While it has been noted that ackermann steering closely resembles the real-world dynamics of a car, it is suitable to some degree of conformity and costs of construction and use for making lightweight autonomous robots made with very few resources.

### **Differential Drive System**

On the other hand, the differential drive system provides a much more simple alternative for the self-parking robot. The differential drive consists of two wheels driven independently of each other located on either side of the chassis and are balanced with the addition of casters or rollers. A motor drives each wheel allowing the robot to move forward, backward, and turn, including turning in place, by controlling the speed and direction of left and right motor rotation. Therefore, a differential drive fits the simplicity and ease of implementation with the TT gear motors and Arduino Uno used in this project, and this system was chosen as the drive system for the self-parking robot.

## **Objective**

The main objective of this project is to develop a low-cost autonomous self-parking robot that can identify a designated parking spot using a color sensor and navigate into the spot without any human intervention. The robot will use sensing, actuation and control via the Arduino Uno platform. The project is intended to demonstrate the application of robotics in real-world automotive settings such as autonomous parking. Additional goals are to evaluate the performance of differential drive locomotion in constrained environments, and demonstrate the feasibility of using color to detect parking spots, and develop a low-cost prototype to demonstrate an example of a self-parking robot.

## **Components Used**

The autonomous self-parking robot is assembled with simple electronic and mechanical components, which work together to provide motion, sensing, and decision-making intelligence. For the hardware design, the Arduino Uno serves as the main controller, the motor driver shield for providing power to the four TT gear motors and wheels, a servo motor adjustment of the sensors for the correct orientation, and an ultrasonic sensor as a means of sensing to detect obstacles. It is powered through two 18650 Li-ion batteries, and a TCS3200 sensor has been implemented to detect the color of the designated parking zones. While each of these components are important for the robot's operation, three components are particularly important to their function and will be talked more about below..

### **3.1 Arduino Uno**

The Arduino Uno is the main microcontroller of the robot, handling sensor inputs and controlling algorithms we wrote in C++. It offers digital and analog input/output pins, as well as serial communication, all of which enable it to work with various sensors and actuators. It's simple, has a large community backing, and programming and prototyping with it are easy and can be trusted in robotics applications. In this project, the Arduino Uno works with the color sensor to detect parking spots, the ultrasonic sensor to avoid obstacles, and the motor driver to actuate the wheels.

### **3.3 Ultrasonic Sensor**

The ultrasonic sensor is an important element for obstacle recognition and distance measurement. It operates by sending out ultrasonic waves and calculating the time it takes for the reflection from an object to return back to the sensor. The Arduino transforms timing data into distance measurements. In the robot, the ultrasonic sensor prevents the robot from crashing when driving and safely assists with parking. As a result of being mounted on the servo, the ultrasonic sensor can scan a wider range of angles, contributing to greater spatial awareness.

### **3.4 Color Sensor**

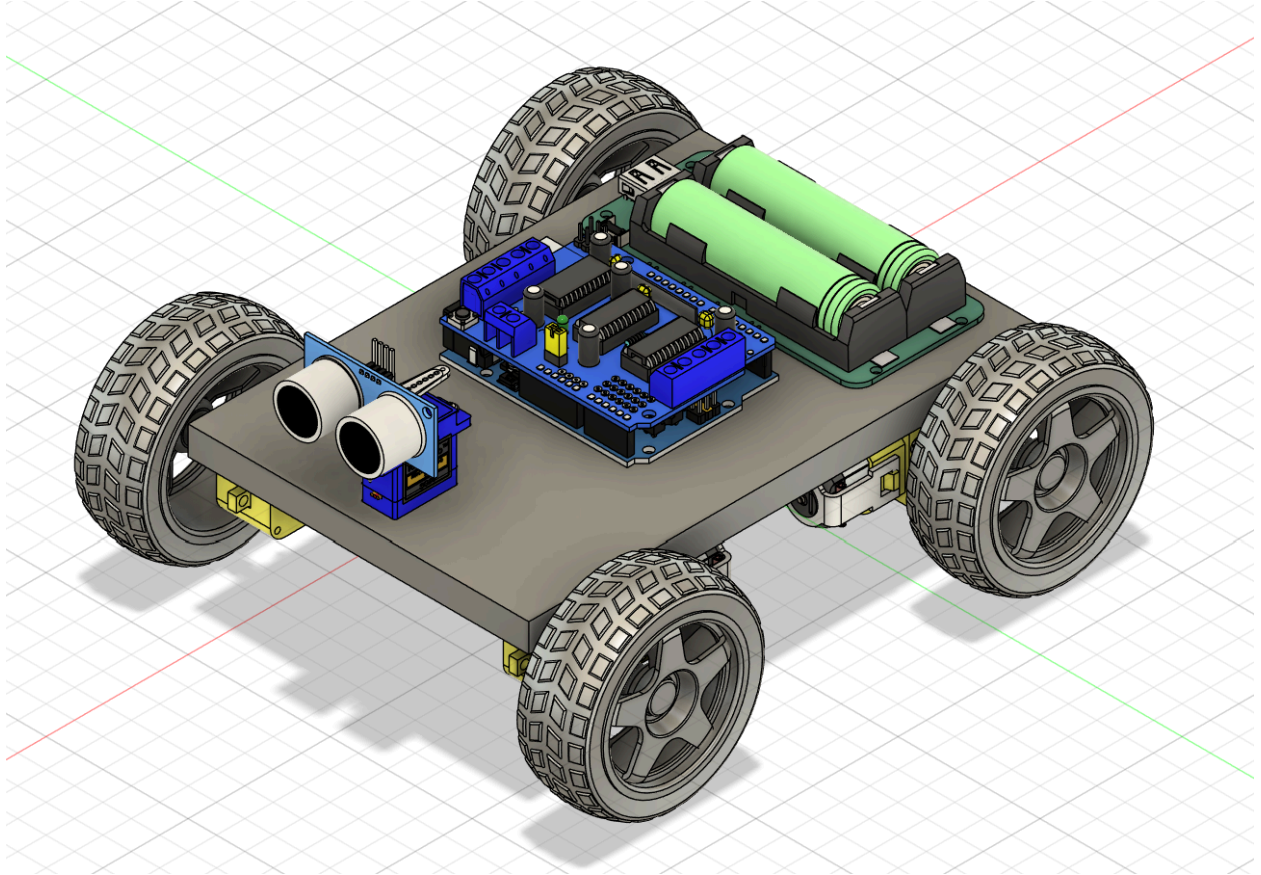
A color sensor (TCS3200) is used to assess the color of the surface underneath the robot, an important feature to identify specified parking areas that are identified by color. The sensor does this by filtering light through red, green, and blue photodiodes and generating a frequency representing intensity of the detected color. The Arduino reads this frequency and assigns a color, which will start the parking sequence when the appropriate color is detected. Because the robot uses color-based detection, it is very accurate in distinguishing between the regular surface and marked parking spaces.

### **3.5 SG 90 Servo Motor**

The SG90 servo motor is an essential electromechanical component in this project, specifically utilized to provide directional control and expand the spatial awareness of the ultrasonic sensor. Unlike the TT gear motors that provide continuous rotation for locomotion, the servo motor allows for precise, limited-angle rotation, typically over a 0 to 180 degrees range, which is ideal for sensor scanning.

The servo's primary function is to act as a pan mechanism for the ultrasonic sensor, allowing the sensor to quickly and accurately adjust its orientation. By rotating the sensor, the robot can scan a significantly wider range of angles than if the sensor were fixed, thus contributing to greater spatial awareness and improving the accuracy of obstacle avoidance.

## CAD Drawing





## **Methodology**

The project employs a systematic approach starting with the conceptual design and component selection. The microcontroller used is Arduino Uno, programmed in C++ to interpret sensor inputs and control motors. The TCS3200 color sensor detects the markings of the parking spots, and the ultrasonic sensor allows for a detection of nearby obstacles while driving. The drive system consists of four TT gear motors for differential drive, allowing the car to drive forward, backward, and rotate. Differential drive has been thoroughly talked about in the introduction of this report. A motor driver shield controls the power and motors, while a servo motor lets the ultrasonic sensor rotate about an axis to scan more of the environment.

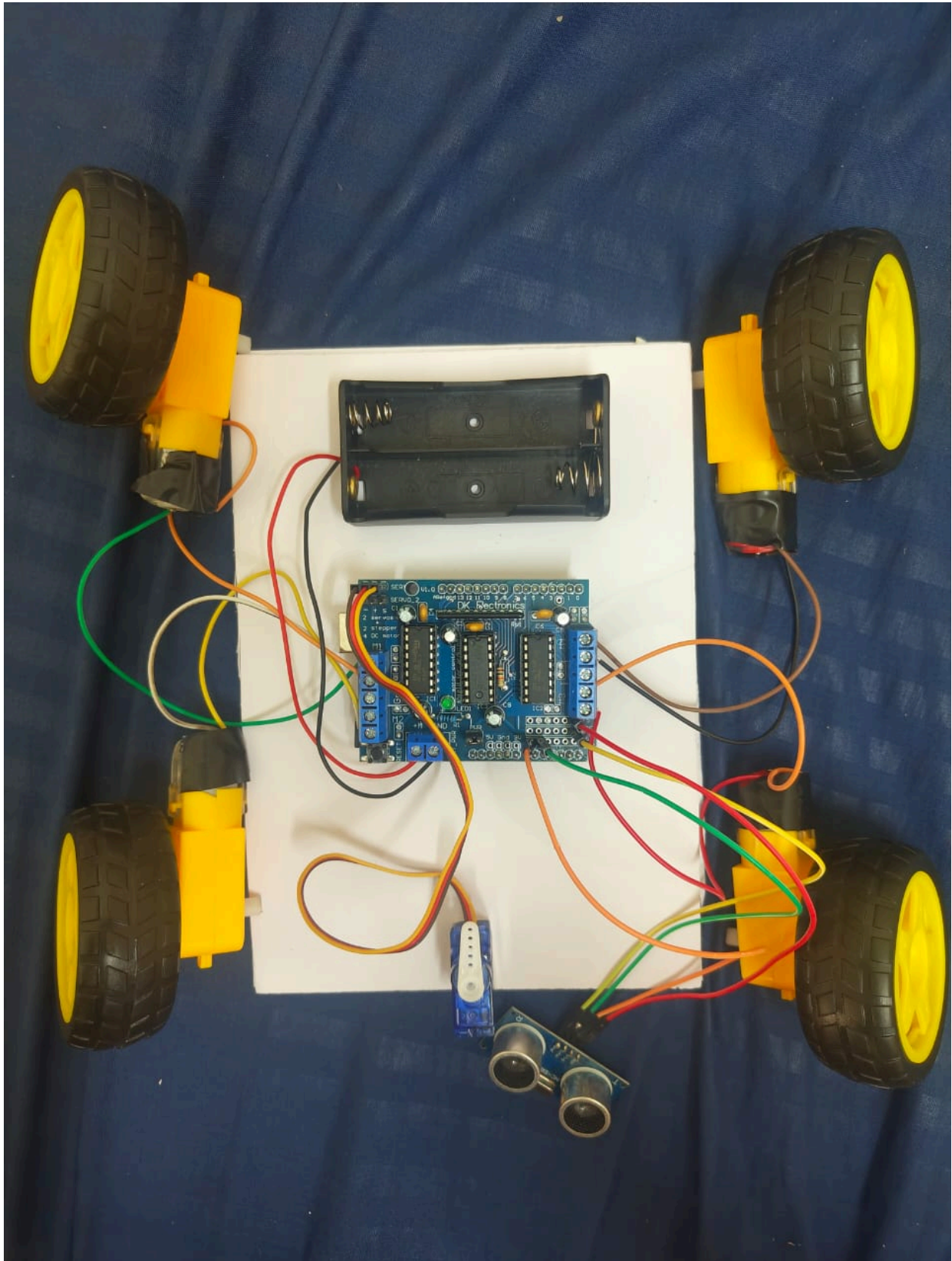
Two 18650 Li-ion batteries power the robot, which makes it portable and gives it decent time without recharging. The project will use foam board for the chassis to create a stable mounting platform for the sensors and support hardware. The development stage involves assembling the hardware, wiring up the sensors and actuators and implementing the control algorithms in C++. Calibration for the color sensor will be performed so it can recognize the colors of the parking spaces, and the navigation logics will be tested in my dorm room.

## Electric Circuit

| Component Pin (Source)                | → | Destination Component | Destination Pin (Arduino Pin) |
|---------------------------------------|---|-----------------------|-------------------------------|
| Power System                          |   |                       |                               |
| 18650 Li-ion Pack (+)                 | → | Motor Driver Shield   | External Power (EXT_PWR)      |
| 18650 Li-ion Pack (-)                 | → | Motor Driver Shield   | GND                           |
| Locomotion System (4 TT Gear Motors)  |   |                       |                               |
| Left Front Motor                      | → | Motor Driver Shield   | M1 (Terminals)                |
| Right Front Motor                     | → | Motor Driver Shield   | M2 (Terminals)                |
| Left Rear Motor                       | → | Motor Driver Shield   | M3 (Terminals)                |
| Right Rear Motor                      | → | Motor Driver Shield   | M4 (Terminals)                |
| Distance Sensing (HC-SR04 Ultrasonic) |   |                       |                               |
| Ultrasonic VCC                        | → | Arduino/Shield        | 5V                            |
| Ultrasonic GND                        | → | Arduino/Shield        | GND                           |
| Ultrasonic Trig                       | → | Arduino Uno           | Digital Pin 10                |
| Ultrasonic Echo                       | → | Arduino Uno           | Digital Pin 11                |
| Directional Control (SG90 Servo)      |   |                       |                               |
| SG90 Power (Red)                      | → | Arduino/Shield        | 5V                            |
| SG90 Ground (Brown)                   | → | Arduino/Shield        | GND                           |
| SG90 Signal (Orange)                  | → | Arduino Uno           | Digital Pin 9                 |

|                                             |   |                |               |
|---------------------------------------------|---|----------------|---------------|
| Parking Spot Sensing (TCS3200 Color Sensor) |   |                |               |
| TCS3200 VCC                                 | → | Arduino/Shield | 5V            |
| TCS3200 GND                                 | → | Arduino/Shield | GND           |
| TCS3200 S0 / S1                             | → | Arduino Uno    | D4 / D5       |
| TCS3200 S2 / S3                             | → | Arduino Uno    | D6 / D7       |
| TCS3200 OUT                                 | → | Arduino Uno    | Digital Pin 2 |

## Circuit Picture



## **Python Code Explanation**

The program operates in two main modes: Obstacle Avoidance and Parking. It continuously checks the environment and adjusts the car's movement accordingly within the main loop() function.

### **1. Initialization and Setup**

The code starts by including necessary libraries (AFMotor.h, NewPing.h, Servo.h) and defining constants for sensor pin assignments (A0, A1, 2 through 6) and motor speed (MAX\_SPEED 190). In the setup() function, the servo motor is attached and centered, and the TCS3200 color sensor pins are configured in colorSensorSetup(). Multiple initial calls to readPing() are made to stabilize the ultrasonic sensor.

### **2. Main Loop and Decision Logic**

The loop() function determines the robot's behavior in every cycle:

1. **Parking Check:** It first checks the isParked flag. If true, the robot stops (moveStop()) and exits the loop. This ensures it stays parked once the goal is achieved.
2. **Color Detection:** It calls detectRed(). If this function returns true (indicating a red surface, like a parking spot, is found), the isParked flag is set to true, the car stops, and a message is printed to the serial monitor.
3. **Obstacle Avoidance:** If the robot is not parked and the current distance is  $\leq 15$  cm, the Obstacle Avoidance logic executes:
  - The car stops (moveStop()) and backs up slightly (moveBackward()).
  - It scans the environment by calling lookRight() and lookLeft() (which move the servo-mounted ultrasonic sensor).
  - It compares the two distances and turns towards the direction with more clearance using turnRight() or turnLeft().
4. **Forward Movement:** If no obstacle is too close and no red is detected, the car continues forward (moveForward()).

### 3. Motor and Movement Control

The DC motors are controlled via the AFMotor library. The `moveForward()` and `moveBackward()` functions include a speed ramp-up using a for loop to gradually increase motor speed from 0 to `MAX_SPEED`. This is a common practice to reduce electrical strain on the battery and components. `turnRight()` and `turnLeft()` implement turning by running the motors on opposite sides in different directions, causing the car to pivot in place for a brief moment.

### 4. Sensor Functionality

- Ultrasonic Sensor (`readPing()`): Uses the NewPing library to measure distance in centimeters. It handles readings of 0 (sensor error/out of range) by assigning a high value (250 cm) to prevent the robot from reacting to erroneous data.
- Color Sensor (`readColor()` and `detectRed()`): The TCS3200 sensor is controlled by setting pins S2 and S3 to filter for Red, Green, or Blue light. The `pulseIn(OUT, LOW)` function measures the frequency of the selected color component. A lower `pulseIn` duration means a higher frequency is detected (i.e., more of that color is present).
  - `detectRed()` performs a simple check: if the Red reading is below a set `RED_THRESHOLD` and is also the lowest reading compared to Green and Blue, it assumes the surface is Red.

## **Conclusion**

This project successfully completed the design and implementation phases for an autonomous, obstacle-avoiding vehicle with specialized parking capabilities. The Arduino Uno platform serves as the foundation for the control system, integrating movement commands with real-time environmental data acquisition.

The robot's design features a robust obstacle avoidance subsystem, utilizing an Ultrasonic Sensor (HC-SR04) mounted on a Servo Motor to enable environmental scanning and dynamic path planning. Crucially, the final design incorporates a TCS3200 Color Sensor to address the autonomous parking objective. The control logic specifies that upon detecting the designated red color threshold, the vehicle transitions from its general pathfinding mode to a stationary, parked state.

The comprehensive project structure includes the conceptualization and selection of all necessary Components Used (Arduino, DC Motors, Sensors), detailed CAD Drawings, and the development of the Electric Circuit design. A clear Methodology guided the entire development process. The C++ Arduino logic and the accompanying Python Code Explanation provide a complete outline of the control algorithms, successfully validating the integration of the hardware and the intelligent control software. The project confirms the feasibility of the proposed architecture for low-cost, multi-functional mobile robots capable of complex navigation tasks.

